Hands-On Generative AI

NLP: Understanding Language as Data

Day 2 - Session 1

KavinSchool

What We'll Cover

1. What NLP is and where you already use it
2. Tokenization: turning text into tokens (BPE and WordPiece)
3. Vectorization: bag-of-words and its limits
4. Embeddings: dense meaning and similarity search
5. Two local demos: tokenization, bag-of-words vs embeddings
6. Lab 2.1: find similar documents with embeddings

Day 1 Recap

Yesterday you worked with numbers and images:

- Trained a classifier and an image model
- Built a VAE and generated images with diffusion
- Saw that generation is sampling from a learned distribution

Today we turn to text. The first job is getting language into numbers a model can use.

What Is NLP?

Getting computers to work with human language

KavinSchool

Natural Language Processing

NLP is the set of techniques that let software read, interpret, and generate human language.

- **Read:** parse and understand text
- **Interpret:** classify, extract, compare meaning
- **Generate:** write, summarize, translate

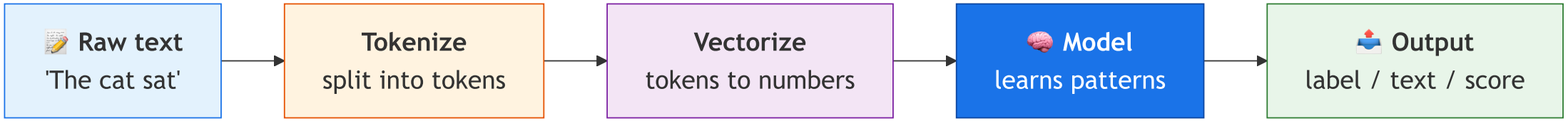
Every LLM (large language model) sits on top of these ideas.

Where You Already Use NLP

Task	Example
Classification	Spam filter, sentiment tags
Extraction	Pull dates and names from a contract
Search	Find the right doc for a question
Translation	English to Japanese
Generation	Autocomplete, chat replies

The Core Problem

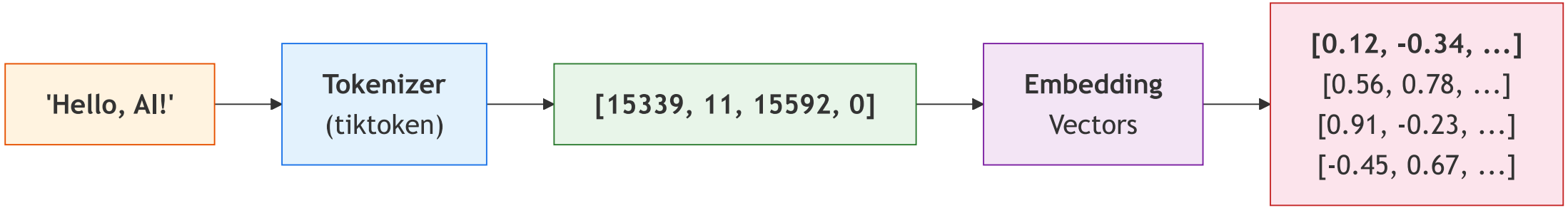
Models work with numbers, not words.



The pipeline is the same across tasks: tokenize, vectorize, then feed a model.

How LLMs see text

LLMs process text as a sequence of integers, not characters or words.



Tokenization

Splitting text into pieces a model can count

KavinSchool

What Is a Token?

A token is a chunk of text the model treats as one unit. It can be a word, a subword, or a few characters.

- "cat" is one token
- "tokenization" often splits into "token" + "ization"
- A rough rule: 1 token is about 4 characters of English

Each token maps to an integer ID the model looks up.

Why Word-Level Splitting Fails

Word-level splitting breaks down fast:

- Vocabularies get huge (every word form is separate)
- New or rare words become unknown tokens
- Typos and made-up words have no entry

Subword tokenization solves all three.

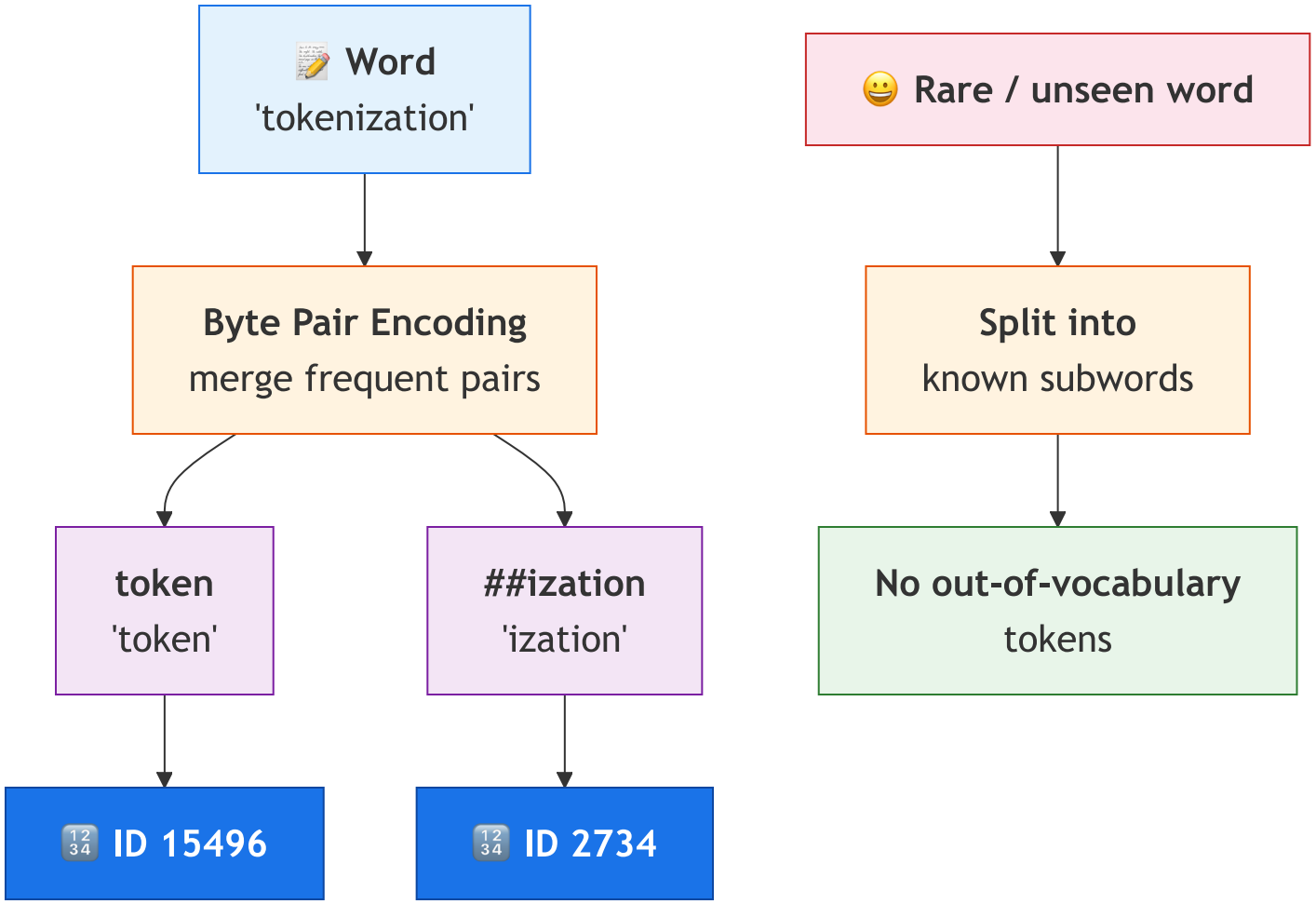
> **NOTE**
>
> Modern LLMs use vocabularies between 50,000 and 128,000 subwords. This strikes a balance between sequence length and embedding matrix size.

Byte Pair Encoding (BPE)

BPE starts from characters and merges the most frequent pairs until it hits a target vocabulary size.

In Practice: A medical LLM might tokenize "acetaminophen" as one token if it saw it often during training, but a general model might split it into "ac", "et", "amino", "phen".

Byte Pair Encoding (BPE)



BPE and WordPiece

Method	Used by	Idea
BPE	GPT (Generative Pre-trained Transformer), Llama	Merge the most frequent character pairs
WordPiece	BERT (Bidirectional Encoder Representations from Transformers)	Merge pairs that most improve likelihood

Both produce subwords, so any word can be built from known pieces. No word is ever truly out of vocabulary.

Tokenization Matters in Practice

- **Cost:** APIs bill per token, so token count is your budget
- **Limits:** context windows are measured in tokens
- **Languages:** some languages use more tokens per word than English
- **Code:** whitespace and symbols each cost tokens

Counting tokens is a habit worth building early.

> **TIP**
>
> Always measure input and output lengths in tokens, not words, when estimating API costs or evaluating context limits.

Demo: Tokenization

Run `day2/demos/demo-tokenization.py`.

- Tokenize the same text with GPT-style BPE and BERT-style WordPiece
- Watch rare words like "acetaminophen" split into subword pieces
- Compare token counts against character and word counts

> Runs locally, no API key. Models count tokens, not words.

Vectorization

Turning tokens into numbers

KavinSchool

Bag-of-Words

The simplest way to vectorize: count how often each word appears, ignore order.

```
"the cat sat"  -> {the: 1, cat: 1, sat: 1}
"the dog sat"  -> {the: 1, dog: 1, sat: 1}
```

Each document becomes a long vector with one slot per vocabulary word.

> **WARNING**
>
> Bag-of-words ignores word order completely. This means sentences with opposite meanings can have identical vectors if they use the same words.

Where Bag-of-Words Falls Short

- **No order:** "dog bites man" and "man bites dog" look identical
- **No meaning:** "car" and "automobile" are unrelated slots
- **Sparse and large:** vectors have thousands of mostly-zero entries

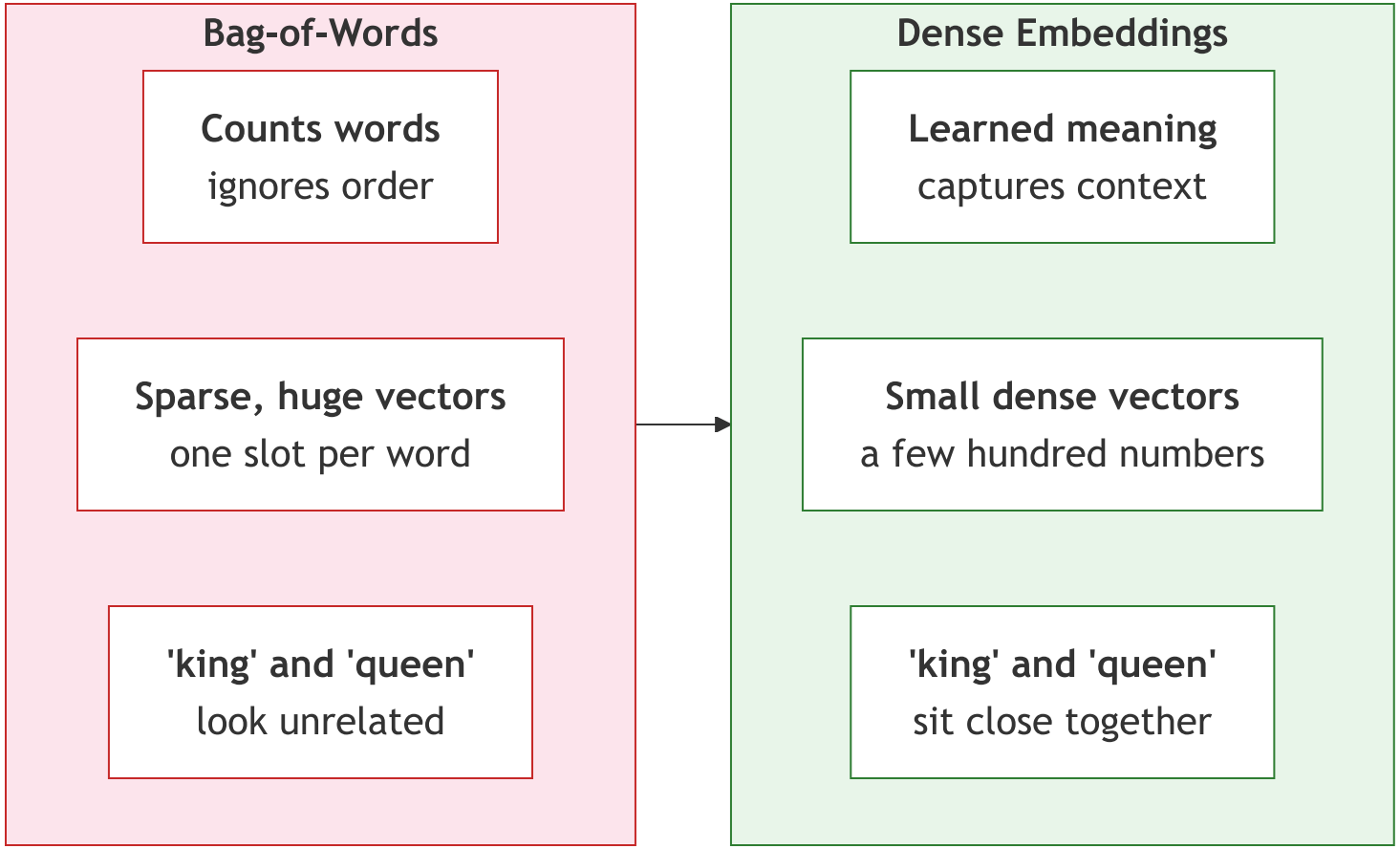
We need vectors that capture meaning rather than raw counts.

Embeddings

Dense vectors that capture meaning

KavinSchool

From Counts to Meaning



What an Embedding Is

An embedding is a dense vector, a few hundred numbers, learned so that similar meanings land near each other.

- "king" and "queen" sit close together
- "king" and "banana" sit far apart
- The vector is learned from context in huge amounts of text

Similar text, similar vector.

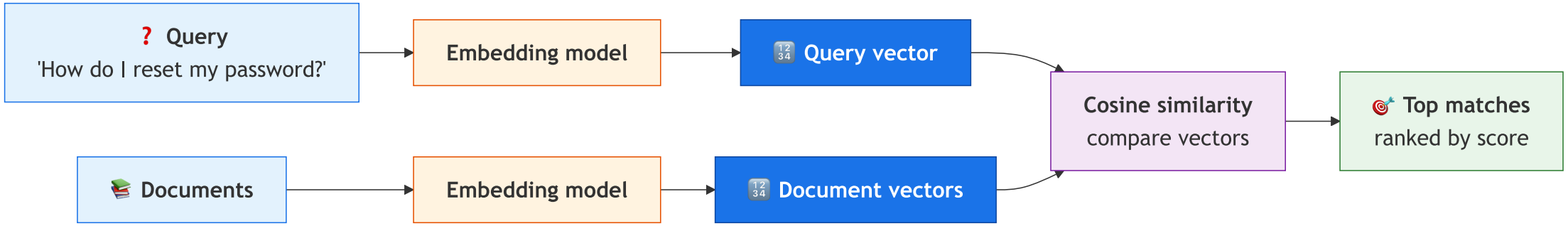
Measuring Similarity

Cosine similarity measures the angle between two vectors.

- Value near 1: vectors point the same way (very similar)
- Value near 0: unrelated
- Value near -1: opposite

It ignores length and focuses on direction, which is what we want for meaning.

Similarity Search



Embed the query and the documents, then rank documents by cosine similarity to the query.

Intuit Scenario: TurboTax Help Search

TurboTax uses embeddings to connect user questions to relevant help articles.

- **The challenge:** Users type "can I write off my home office?" but the article says "deducting home workspace expenses."
- **The solution:** Both phrases map to similar dense vectors.
- **The outcome:** Similarity search retrieves the right article even without exact keyword matches.

Why This Matters

Similarity search over embeddings is the engine behind:

- Semantic search (find by meaning, not keywords)
- Recommendations ("more like this")
- Retrieval-augmented generation, which you build on Day 3

Get comfortable with embeddings now and RAG will feel familiar later.

Demo: Bag-of-Words vs Embeddings

Run `day2/demos/demo-bow-vs-embeddings.py`.

- See "dog bites man" and "man bites dog" get identical bag-of-words vectors
- Watch embeddings score "car" and "automobile" sentences as similar
- Rank documents against a query by cosine similarity

> Runs locally, no API key. Meaning lives in the embedding, not the word count.

Breakout Lab 2.1

Find similar documents with embeddings

KavinSchool

Lab 2.1: Find Similar Documents

Open `day2/lab2_1_similarity/start.ipynb` in Colab.

- Load a small set of documents
- Embed them with a sentence-transformers model
- Rank documents by cosine similarity to a query
- Compare semantic search against keyword matching

> Stretch goal: cluster the documents or try a second embedding model and compare.

Key Takeaways

- [x] NLP turns language into numbers so models can work with it
- [x] Tokenization splits text into subwords, so no word is out of vocabulary
- [x] Bag-of-words counts words but loses order and meaning
- [x] Embeddings are dense vectors where similar meanings sit close together
- [x] Cosine similarity over embeddings powers search, recommendations, and RAG

Questions?

Next: Large Language Models

Morning break: 15 minutes

KavinSchool